trace. Just for fun, you can hack irb's source and remove the last two arguments from the call to eval (remember to undo the change afterward):

```
eval(statements, @binding) # , file, line)
```

Run irb exception.rb now, and the exception reports the file and line where eval is called:

```
ZeroDivisionError: divided by 0
  from /Users/nusco/.rvm/rubies/ruby-2.0.0/lib/ruby/2.0.0/irb/workspace.rb:54:in `/'
```

This kind of hacking of the stack trace is especially useful when you write Code Processors—but consider using it everywhere you evaluate a *String of Code (141)* so you can get a better stack trace in case of an exception.

Strings of Code vs. Blocks

In *Kernel#eval, on page 141*, you learned that eval is a special case in the eval* family: it evaluates a *String of Code (141)* instead of a block, like both class_eval and instance_eval do. However, this is not the whole truth. Although it's true that eval always requires a string, instance_eval and class_eval can take either a String of Code or a block.

This shouldn't come as a big surprise. After all, code in a string is not that different from code in a block. Strings of Code can even access local variables like blocks do:

```
array = ['a', 'b', 'c']
x = 'd'
array.instance_eval "self[1] = x"

array   # => ["a", "d", "c"]
```

Because a block and a String of Code are so similar, in many cases you have the option of using either one. Which one should you choose? The short answer is that you should probably avoid Strings of Code whenever you have an alternative. Let's see why.

The Trouble with eval()

Strings of Code are powerful, no doubt about that. But with great power comes great responsibility—and danger.

To start with, Strings of Code don't always play well with your editor's syntax coloring and autocompletion. Even when they *do* get along with everyone, Strings of Code tend to be difficult to read and modify. Also, Ruby won't report a syntax error in a String of Code until that string is evaluated, potentially resulting in brittle programs that fail unexpectedly at runtime.